

Zero to Hero Study Handbook: credit-risk-decisioning-system

Module 1: Foundations & Architecture

What this project does

This repository builds an end-to-end credit risk decisioning workflow for binary default prediction and policy banding into **approve**, **manual_review**, or **reject** decisions. The training workflow is orchestrated in the notebook `credit_risk_decisioning_system.ipynb`, and scoring is served through a FastAPI app in `app/main.py`.

Primary use cases from repo contents: - Train and compare multiple model-selection tracks (LazyPredict discovery, manual engineering, FLAML, PyCaret) using Home Credit data. - Generate deployable artifacts (`.joblib` model + preprocessor + threshold JSON) for scoring. - Serve single-applicant scoring over HTTP with decision bands.

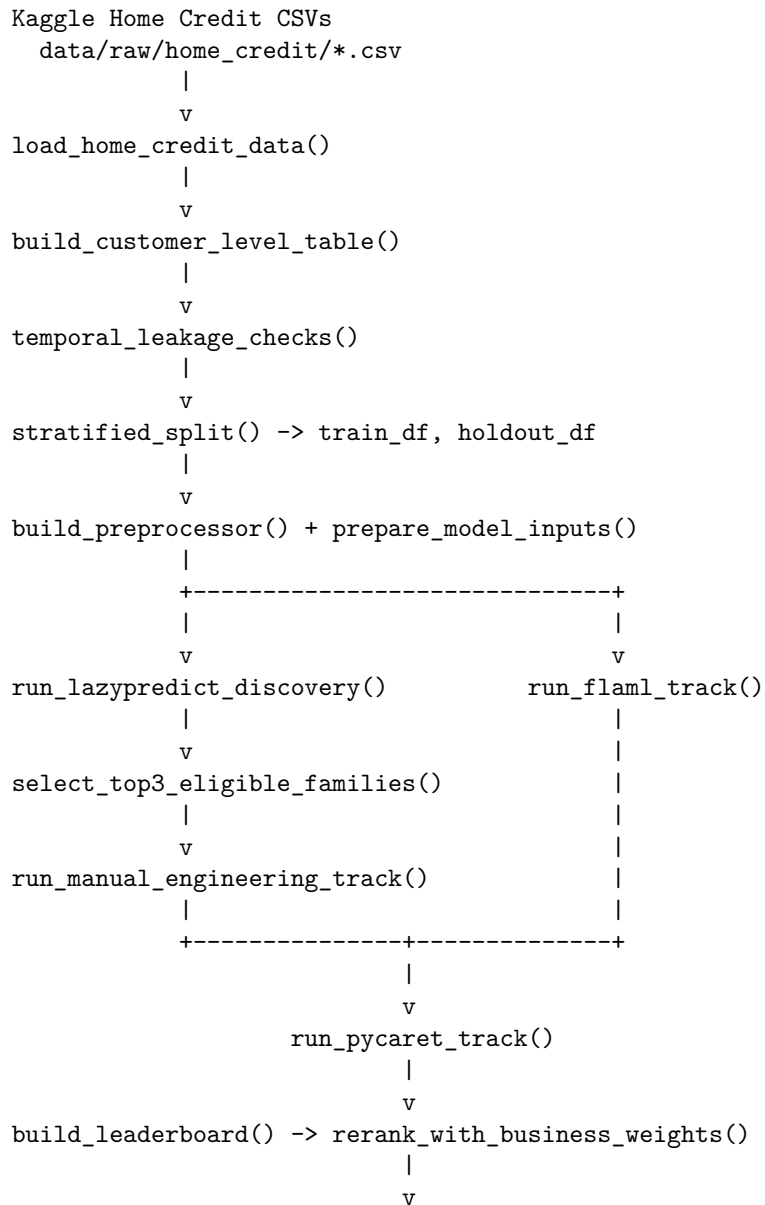
Core paradigms and patterns actually used

1. **Functional pipeline modules** Each stage is a function, mostly in `src/` (`load_home_credit_data`, `build_preprocessor`, `run_flaml_track`, `build_leaderboard`, etc.), then composed in notebook cells.
2. **Notebook orchestration over library modules** Business workflow order is controlled in the notebook, while reusable logic stays in modules. This keeps experimentation in notebook cells and implementation in Python files.
3. **Multi-track model selection** `src/modeling.py` implements four tracks:
 - Discovery: `run_lazypredict_discovery`
 - Manual top-3 engineering: `run_manual_engineering_track`
 - AutoML challenger: `run_flaml_track`
 - Experiment-lab challenger: `run_pycaret_track`
4. **Artifact-based deployment boundary** Training and serving are decoupled through files:
 - `artifacts/credit_risk_model.joblib`
 - `artifacts/credit_risk_preprocessor.joblib`
 - `artifacts/decision_thresholds.json`
5. **Policy thresholding pattern** Raw default probabilities are mapped to actions using two thresholds (approve/reject). This appears both in training logic (`assign_decision_band`) and serving logic (`_decision_band`).

Architecture: components and interaction

Main components: - **Data ingestion and table construction:** `src/data_prep.py`
- **Preprocessing pipeline:** `src/features.py` - **Model training/selection**
tracks: `src/modeling.py` - **Leaderboard/ranking:** `src/evaluation.py` -
Serving API: `app/main.py` - **Operational scripts:** `scripts/`

ASCII architecture diagram (main path):



leaderboard_credit_risk.csv + best model selection

|

v

save_inference_bundle()

-> credit_risk_model.joblib

-> credit_risk_preprocessor.joblib

-> decision_thresholds.json

|

v

FastAPI /score endpoint (app/main.py)

Module 2: Repository Map

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
README.md	Project scope, setup commands, artifact expectations	N/A	Dataset path data/raw/home_credit , artifact names in artifacts/
pyproject.toml	Dependency and Python/runtime constraints	N/A	requires-python = "==3.12.10", project dependencies, [tool.uv].package = false
credit_risk_decision_system.py	Modeling system, orchestration path	calls all src pipeline functions	SEED , RAW_DIR , ARTIFACT_DIR , PROJECT_NAME
credit_risk_decision_system.py	Cloning system, run outputs and logs	See src/ and nb notebook	Concrete output examples (class balance, matrix shapes, rankings)
src/data_prep.py	Load raw tables, aggregate by customer, leakage guards, train/holdout split	load_home_credit_data , build_customer_level_features , temporal_leakage_checks , stratified_split	RAW_TABLES , sample_frac , target_col , test_size
src/features.py	Build and apply sklearn preprocessing graph	build_preprocessor , prepare_model_inputs	SimpleImputer strategies, StandardScaler(with_mean=False) , OneHotEncoder(handle_unknown="ignore")

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
src/modeling.py	Candidate families, training tracks, calibration, threshold utility optimization, artifact saving	make_estimator, run_lazypredict_ddiscovery, select_top3_eligible_candidates, run_manual_engineer25/0.65, run_flaml_track, FLAML, run_pycaret_tracktime_budget, save_inference_bundle	FAMILY_ALIASES, defaults, thresholds, FLAML, time_budget, utility weights (7,-2,-5)
src/evaluation.py	Unified leaderboard schema and ranking formulas	build_leaderboard, rerank_with_business_weights	REQUIRED_COLUMNS, defaults (0.62/0.25/-0.08/-0.05)
app/main.py	HTTP scoring service and decision band inference	ScoreRequest, _load_thresholds, _decision_band, health, score	MODEL_PATH, PREPROCESSOR_PATH, THRESHOLD_PATH, default thresholds 0.25/0.65
scripts/setup_env.sh	Bootstrap local environment	Bash script	uv venv --python 3.12.10, uv sync
scripts/download_data.sh	Download and unzip Kaggle competition data	Bash script	Creates data/raw/home_credit, downloads home-credit-default-risk.zip
scripts/run_notebook.sh	Starts notebook runtime	Bash script	Launches jupyter lab credit_risk_decisioning_system.ipynb

Module 3: Core Execution Flows

Flow A: Training and artifact generation (Notebook main path)

Primary entrypoint: - credit_risk_decisioning_system.ipynb

Step-by-step:

1. Initialize constants and paths:
 - SEED = 42
 - RAW_DIR = Path('data/raw/home_credit')
 - ARTIFACT_DIR = Path('artifacts')
2. Load source tables:

- `tables = load_home_credit_data(RAW_DIR, sample_frac=0.05, random_state=SEED)`
 - Returned type: `Dict[str, pd.DataFrame]` with keys from `CORE_TABLES`:
 - `application_train`
 - `bureau`
 - `previous_application`
 - `POS_CASH_balance`
 - `installments_payments`
 - `credit_card_balance`
3. Build customer-level modeling table:
- `customer_df = build_customer_level_table(tables)`
 - Merge pattern:
 - Base is `application_train`
 - Numeric aggregations over auxiliary tables grouped on `SK_ID_CURR`
 - Linked tables use `SK_ID_PREV` -> `SK_ID_CURR` mapping
 - Output includes `TARGET` plus engineered aggregate columns like `bureau_<col>_<stat>`.
4. Apply leakage and sparsity checks:
- `customer_df = temporal_leakage_checks(customer_df)`
 - Drops:
 - Any column starting with `target` (case-insensitive) except exact `TARGET`
 - `SK_ID_PREV`, `SK_ID_BUREAU` if present
 - Columns with missing ratio > 0.995
5. Stratified split and preprocessing:
- `train_df, holdout_df = stratified_split(customer_df, target_col='TARGET', random_state=SEED)`
 - `preprocessor = build_preprocessor(train_df.drop(columns=['TARGET']))`
 - `X_train, X_holdout, y_train, y_holdout = prepare_model_inputs(...)`
 - Encoded output shape is sparse matrix-like (from `ColumnTransformer` + `OneHot`), plus integer target vectors.
6. Discovery track:
- `lazy_table = run_lazypredict_discovery(X_train, X_holdout, y_train, y_holdout)`
 - Output DataFrame columns (when present):
 - Model, Accuracy, Balanced Accuracy, ROC AUC, F1 Score, Time Taken
7. Eligibility filter:
- `eligible_table, top3_families = select_top3_eligible_families(...)`
 - Eligibility rule in code:
 - `stable = brier < 0.25`

- `eligible = stable and train_time < 600`
- `top3_families` type: `list[str]` (up to 3 families).

8. Manual engineering track:

- `manual_results, manual_models = run_manual_engineering_track(...)`
- `manual_results` includes:
 - `model_name`, `pr_auc`, `roc_auc`, `brier`
 - `train_time_sec`, `infer_latency_ms`, `p95_latency_ms`
 - `optimized_threshold`, `policy_utility`
 - `holdout_scores` (probability array), `calibration_metric`
- `manual_models` is a mapping: `{family_name: fitted_model_object}`.

9. FLAML and PyCaret tracks:

- `flaml_result = run_flaml_track(...)`
 - Dict keys include `model_name`, `pr_auc`, `roc_auc`, `brier`, `best_config`, `best_loss`, `time_budget`.
- `pycaret_result = run_pycaret_track(...)`
 - Dict includes metrics plus status ("ok" or "failed").

10. Unified ranking:

- `leaderboard = build_leaderboard(...)`
- `leaderboard = rerank_with_business_weights(leaderboard)`
- Persisted to `artifacts/leaderboard_credit_risk.csv`.

11. Deployable bundle save:

- Winner selection: `winner = leaderboard.sort_values('final_rank').iloc[0]['model_name']`
- If winner is in `manual_models`, notebook calls `save_inference_bundle(...)`.
- Persisted files:
 - `artifacts/credit_risk_model.joblib`
 - `artifacts/credit_risk_preprocessor.joblib`
 - `artifacts/decision_thresholds.json`

Short code fragment (from notebook orchestration):

```
leaderboard = build_leaderboard(
    project_name=PROJECT_NAME,
    task_type='binary_classification',
    lazy_results=eligible_table,
    manual_results>manual_results,
    flaml_result=flaml_result,
    pycaret_result=pycaret_result,
    baseline_rows=baseline_rows,
    manual_model_objects>manual_models,
)
leaderboard = rerank_with_business_weights(leaderboard)
leaderboard.to_csv(ARTIFACT_DIR / 'leaderboard_credit_risk.csv', index=False)
```

Flow B: Online scoring API request/response

Entrypoint: - `app/main.py`

Endpoints: - `GET /health` returns `{"status": "ok"}` - `POST /score` scores one applicant payload

Request model: - `ScoreRequest` with one field: - `features: Dict[str, Any]`
- Expected shape is a flat key-value map (the API does not hardcode feature names; it passes the dict into a one-row `DataFrame`).

`/score` internal sequence: 1. Verify artifact files exist; otherwise raise HTTP 404 with message "Model artifacts not found. Train notebook first." 2. Load model and preprocessor via `joblib.load`. 3. Convert input dict into `payload_df = pd.DataFrame([request.features])`. 4. Transform with `preprocessor.transform(payload_df)`. 5. Compute default probability `model.predict_proba(x)[:, 1][0]`. 6. Load thresholds from `decision_thresholds.json`; fallback defaults are 0.25 and 0.65. 7. Map score to `approve/manual_review/reject`.

Response shape:

```
{
  "probability_of_default": 0.0,
  "decision_band": "approve",
  "thresholds": {
    "approve_threshold": 0.25,
    "reject_threshold": 0.65
  }
}
```

Decision policy logic:

```
if score < approve_thr:
    return "approve"
if score >= reject_thr:
    return "reject"
return "manual_review"
```

Flow C: Scripted operational flow

Scripts and order: 1. `scripts/setup_env.sh` - Creates `.venv` with Python 3.12.10 - Runs `uv sync` 2. `scripts/download_data.sh` - Downloads Kaggle competition ZIP - Unzips under `data/raw/home_credit` 3. `scripts/run_notebook.sh` - Activates environment - Starts Jupyter Lab with the project notebook

Module 4: Setup & Run Guide

Dependency and runtime baseline

From `pyproject.toml`: - Python must be `==3.12.10` - Package manager workflow is `uv` - Core runtime libs include: - Data/ML: `pandas`, `numpy`, `scikit-learn`, `xgboost`, `lightgbm`, `catboost`, `lazypredict`, `flaml`, `pycaret` - Serving: `fastapi`, `uvicorn` - Notebook: `jupyter`, `ipykernel`

Fresh-machine setup commands

```
cd /home/ahmad/AI/Github/risk-fraud-aml-systems/credit-risk-decisioning-system
bash scripts/setup_env.sh
```

Data acquisition commands

```
cd /home/ahmad/AI/Github/risk-fraud-aml-systems/credit-risk-decisioning-system
bash scripts/download_data.sh
```

Equivalent direct commands from `README.md/script`:

```
mkdir -p data/raw/home_credit
kaggle competitions download -c home-credit-default-risk -p data/raw/home_credit
unzip -o data/raw/home_credit/home-credit-default-risk.zip -d data/raw/home_credit
```

Run commands in this repo

Notebook (documented):

```
bash scripts/run_notebook.sh
```

or

```
source .venv/bin/activate
jupyter lab credit_risk_decisioning_system.ipynb
```

API serving: - The repo defines the FastAPI app object in `app/main.py`, but does not include a dedicated API startup script/command in `README.md` or `scripts/`. - Runtime prerequisite for scoring API is the artifact trio in `artifacts/`.

Required environment variables and config files

Application code requirements: - No `.env` file is referenced in repository code. - No explicit environment variable keys are read in `src/` or `app/main.py`.

Operational requirements outside app code: - Dataset download script uses `kaggle competitions download`, so Kaggle CLI authentication must be configured in your environment. This requirement is implied by script usage, not by explicit `env-key` reads in repo Python code.

Migrations/seeding/external-service initialization

- Database migrations: none (no DB migration tooling or schemas in repo).
- Data seeding equivalent: Kaggle dataset download + unzip into `data/raw/home_credit`.
- Artifact seeding for serving: produced by notebook via `save_inference_bundle(...)`.

Module 5: Study Plan & Practice Exercises

Ordered study plan for a new learner

1. Read `README.md` and `pyproject.toml` to understand goal, dependencies, and expected outputs.
2. Read the notebook `credit_risk_decisioning_system.ipynb` end-to-end to see orchestration order.
3. Deep dive `src/data_prep.py` for dataset schema expectations and leakage controls.
4. Study `src/features.py` to understand preprocessing transformations and encoded matrix creation.
5. Study `src/modeling.py` carefully; this is the most logic-dense module.
6. Study ranking and schema logic in `src/evaluation.py`.
7. Read serving contract in `app/main.py`.
8. Use `credit_risk_decisioning_system.executed.ipynb` to inspect a concrete sample run outcome.

Practice exercises

1. **Trace the minimum required source columns.** Question: Which columns are strictly required for the full feature pipeline to work through `build_customer_level_table` and why?
2. **Explain leakage defenses.** Question: What exactly does `temporal_leakage_checks` remove, and which logic is name-based vs sparsity-based?
3. **Decode engineered feature naming.** Question: If you see a feature named `bureau_AMT_CREDIT_SUM_mean`, which function created it and by what rule?
4. **Reconstruct manual-track eligibility.** Question: From `select_top3_eligible_families`, list the exact eligibility criteria and stopping rule for model-family selection.
5. **Understand policy optimization.** Question: How does `optimize_operating_threshold` evaluate a threshold, and what business tradeoff is encoded by its utility function?
6. **Compare ranking formulas.** Question: How does ranking in `build_leaderboard` differ from `rerank_with_business_weights`?

7. **Rebuild the API contract from code only.** Question: Write the exact request/response schema for `POST /score`, including error behavior when artifacts are missing.
8. **Deployment artifact reasoning.** Question: Why are both `credit_risk_model.joblib` and `credit_risk_preprocessor.joblib` needed by the API instead of model-only deployment?

Solution outlines

1. `TARGET` must exist in `application_train` (`build_customer_level_table` raises if missing). `SK_ID_CURR` is required for merging aggregated tables. `SK_ID_PREV` is required to link `POS_CASH_balance`, `installments_payments`, and `credit_card_balance` via `previous_application`.
2. Name-based removal: columns where lowercase name starts with `target` except exact `TARGET`, plus `SK_ID_PREV` and `SK_ID_BUREAU` if present. Sparsity-based removal: any column with missing ratio > 0.995 .
3. `_aggregate_numeric` groups by a key, computes `mean/max/min/std`, and renames as `"{prefix}_{column}_{stat}"`. Prefix `bureau` means the aggregate came from `bureau.csv`.
4. Eligibility is `brier < 0.25` and `train_time < 600`. Families are selected in lazy-model ranking order, deduplicated by family alias, and iteration stops once 3 eligible families are collected.
5. Threshold candidates are 0.1 to 0.9 with 81 evenly spaced points. Utility is $(7 * TP) - (2 * FP) - (5 * FN)$. This weights false negatives as more costly than false positives.
6. `build_leaderboard` uses fixed formula $0.62 * \text{primary} + 0.25 * \text{secondary} - 0.08 * \text{tertiary} - 0.05 * \text{calibration}$. `rerank_with_business_weights` exposes tunable weights and adds a normalized p95 latency penalty.
7. Request body is `{"features": {<flat feature key>: <value>, ...}}`. Success response includes `probability_of_default`, `decision_band`, and `thresholds`. If model/preprocessor artifacts are absent, API returns HTTP 404 with detail `"Model artifacts not found. Train notebook first."`.
8. The model expects transformed feature space from the fitted preprocessing graph (imputation, scaling, one-hot encoding). Without the persisted preprocessor, live payloads cannot be converted into the same training feature matrix.

Verification Checklist

Use this checklist before claiming mastery:

- ☐ I can explain the full notebook flow from data loading to artifact saving without looking at the code.
- ☐ I can name the exact `CORE_TABLES` loaded by `load_home_credit_data`.
- ☐ I can explain how auxiliary tables are aggregated and merged into the customer-level table.
- ☐ I can explain every drop condition in `temporal_leakage_checks`.
- ☐ I can describe the preprocessing pipeline for numeric and categorical columns.
- ☐ I can explain how top-3 eligible families are selected and why some families are filtered.
- ☐ I can explain threshold utility optimization and decision-band assignment logic.
- ☐ I can reconstruct the leaderboard schema and ranking formulas.
- ☐ I can describe the exact artifact files required by the API.
- ☐ I can describe the `/score` request/response and the 404 failure path.